

Exploring a Synthetic SSc Cohort

Data Understanding, Application, and Data Quality

Semen Leyn

2026-09-09

Agenda

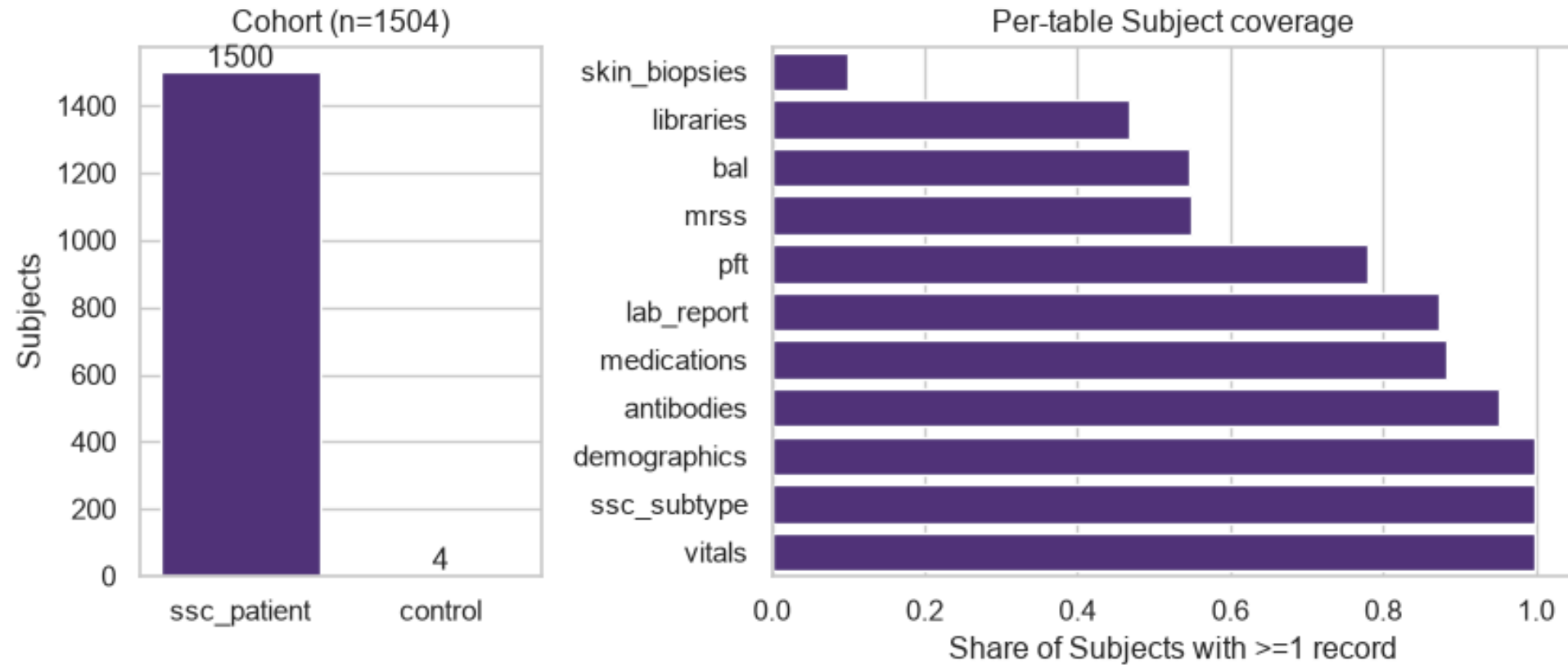
- Understanding the dataset
- The application & visualizations
- Data-quality issues and analytical findings
- Design decisions and tradeoffs
- Future work & discussion

Understanding the Dataset

What's in the box

- 11 CSVs, ~4 MB, synthetic and non-PHI
- A patient/subject identifier under **4 different column names** across files — normalized to one canonical `subject_id` at ingestion
- The **Registry**: `demographics` + `ssc_subtype`, a 1:1, 1,500-row master list — every row `study=SSC-REG`, `diagnosis=SSc`
- Every other table: a **partial-coverage** table of clinical/molecular events for a subset of those 1,500 patients

Population and table coverage



A hidden second cohort

4 subject IDs (`SSC_NORM_0101/0102/0104/0110`) appear in `libraries.csv` — but **never** in the Registry.

- `libraries.csv`'s own comments confirm these are “healthy control sample[s]” used as a comparator arm for the RNA-seq work
- Modeled explicitly as a **Subject supertype** with a **Cohort** attribute (`ssc_patient` vs `control`) — not orphaned/invalid IDs
- A real structural finding, not a data error
- The QC key-linkage check automates this: confirms all 4

Static vs. longitudinal, and a naming trap

- `lab_report`, `vitals`, `mrss` — long-format, one **Observation** abstraction in code (Lab Result / Vital Sign / MRSS stay distinct terms in any user-facing view)
- `bal`, `medications`, `skin_biopsies` — event-based, not scheduled
- `demographics/ssc_subtype` — static, one row per patient

`ssc_subtype.other_dx` looks like a differential diagnosis field — it's actually semicolon-separated

The Application

Streamlit, 4 tabs, SQLite-backed

- **Data & Dictionary** — schema, coverage, QC report link
- **Cohort Overview** — population-level distributions
- **Compare & Discover** — free-form variable picker across any table/level, with a Control Subject overlay toggle
- **Patient Trajectory** — one patient's longitudinal record

Login + at-rest encryption + audit logging gated behind a published demo account (details in Design Decisions).

Data & Dictionary

TODO: screenshot — Data & Dictionary tab

Cohort Overview

TODO: screenshot — Cohort Overview tab

Compare & Discover

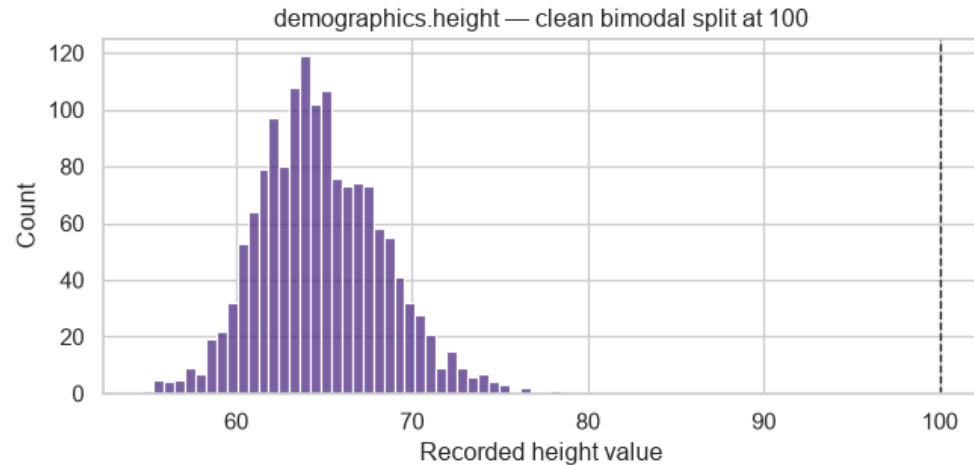
TODO: screenshot — Compare & Discover tab

Patient Trajectory

TODO: screenshot — Patient Trajectory tab

Data-Quality Issues and Analytical Findings

Height recorded in mixed units



1,235 of 1,500 patients in **inches** (54.7-78.5); the other 265 in **centimeters** (141.3-194.7 cm). Fixed at ingestion: cm-scale values converted to inches, raw CSV left untouched.

One high-confidence data-entry error

`demographics.csv` lists `subject_8545`'s weight as **22.5** (impossible for a living adult) — but 5 separate `vitals.csv` visits spanning 2017-2020 all report ~160.6 lbs for the same patient.

Decision: flagged in the QC report, not silently corrected. The vitals-based value is well-evidenced, but `demographics.weight` is a distinct static field with no documented derivation from vitals — overwriting it would assert a number we don't have provenance for.

A synthetic-data placeholder, not a clinical signal

`WEIGHT IN POUND = 160.6` appears 870 times across 195 distinct patients — the next most common value appears only 39 times. Some patients show this exact value on *every* visit across multiple years.

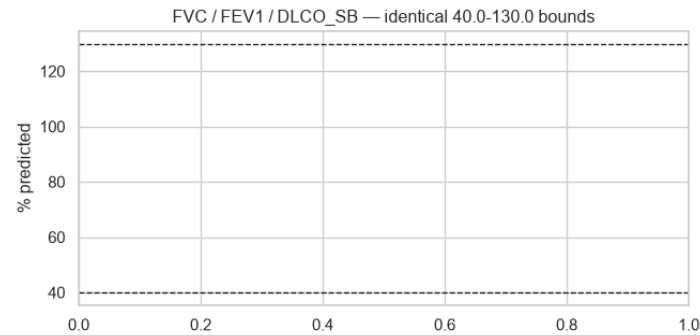
`check_constant_value_across_visits`: flags a value dominating far more *patients* than expected if personal-constant values were spread evenly — 160.6 scores ~106x expectation; the next-highest (BP diastolic=77)

The same pattern in BP and pulse, measured differently

The per-patient check above is blind to a filler used on only *some* of a patient's visits. Looking at each value's share of *every* record instead:

Measure	Value	Share of all records	vs. local background
BP SYSTOLIC	124	25% of 7,848	11.7x
BP DIASTOLIC	77	25% of 7,848	5.7x
PULSE	76	16% of 8,381	4.6x

PFT bounds look like generator clipping



Three physiologically unrelated measures all range **exactly 40.0-130.0% predicted**. A clean guideline-range check here is itself the finding — reported as a result, not treated as “nothing to see.”

The same generic check finds real injected errors

Running `check_guideline_range` against `lab_report.csv` for the first time surfaces isolated, physiologically-impossible values nobody had manually audited:

- `ABSOLUTE LYMPHOCYTES`, `MCH`, `MCHC` — one `9999` sentinel each
- `WBC` — two `999` sentinels
- `HEMOGLOBIN`, `NEUTROPHILS`, `LYMPHOCYTES`, `MCV`, `NM BKR`
`ABSOLUTE NEUTROPHIL` — 1-2 negative values each (impossible for any of those measures)

Medications: names fragmented, doses mostly clean

Only 17 distinct medication strings total — but `CellCept` (192), `mycophenolate mofetil` (211), and `MMF` (189) are the same drug under three names. Merged, the total (592) is nearly 2.5x the next most-prescribed drug (rituximab, 253).

- **Dose:** only 24 distinct strings; a few look like intentionally injected errors — negative doses, `999 tablets daily`, `250 g twice daily` (likely a mg/g slip, ~1000x too high), `500 mL tablets daily` (a

PII-shaped fields in a synthetic dataset

`demographics` has first/last name and birth date;
`mrss/skin_biopsies` have `ENTRY_USER_NAME` (a clinician name) — despite the dataset being confirmed synthetic/non-PHI.

Decision: patient name/DOB suppressed everywhere downstream (`subject_id` + age-at-event only) — a deliberate demonstration of PHI-handling judgment.

`ENTRY_USER_NAME` kept visible: provider-side operational metadata, not what HIPAA governs, and useful for a “does score entry vary by rater?” angle

Design Decisions and Tradeoffs

HIPAA controls, implemented literally

Real login, real encryption at rest, real audit logging — despite the data being confirmed synthetic/non-PHI and the deliverable requiring a public repo + public app link.

Gated by a published demo account, so the controls are **genuine engineering rather than theater**, without blocking reviewer access.

(ADR 0001)

Encrypt the store, not the source CSVs

- Keeps the repo **clone-and-run reproducible** — a reviewer can inspect the actual source data directly
- Still gives a real “encryption at rest” component to explain: the runtime SQLite store the app/notebook actually reads from is encrypted

(ADR 0002)

SQLite with a Subject supertype, not pandas end-to-end

Chosen specifically because it **models the Control Subject / Registry relationship correctly** — cohort membership, not a foreign-key violation to explain away.

The role's own scope ("data modeling across clinical data sources") rewards showing schema design even at a data size where SQL isn't strictly necessary for performance.

(ADR 0005)

Demo secrets committed, against normal practice

The credentials and encryption key are **deliberately public by design** (same reasoning as the HIPAA-literal controls) — keeping them out of git would only add setup friction for reviewers, with no actual security benefit.

Documented explicitly as a demo-only deviation.

(ADR 0004)

Scope triage under a same-day deadline

- Flat package layout over a layered architecture
- Minimal-but-real CI: ruff + mypy + pytest via GitHub Actions
- Targeted tests over exhaustive coverage
- The originally-planned 5th “discovery” app page merged into the group-comparison page once it became clear they were the same free-form explorer

Future Work

Known, deliberately deferred — not bugs. The app is stable, tested, deployed, green.

- Disease-duration x-axis (years-since-onset, not calendar date) — likely washes out cohort-level disease patterns as currently plotted
- Patient Trajectory has no subject header (subtype/antibody/diagnosis date) despite [data/access.py](#) already having it
- Subtype/antibody never faceted against outcome trends (e.g. “MRSS by subtype”) — plausibly the dataset’s intended hypothesis

Thank you — discussion

Repo: (public GitHub link) · App: nw-ssc-de.streamlit.app

Data Understanding, Application, and Data Quality

Appendix

Appendix: HIPAA-style controls, in detail

- **Login:** gated by a published demo account ([demo / SscDemo2026!](#)) — a demonstration of the control, not a real access boundary
- **Encryption at rest:** the runtime SQLite store is built encrypted from the source CSVs at startup; source CSVs themselves are left plaintext for reviewer reproducibility
- **Audit logging:** every login attempt (success/failure) appended to a local [audit.log](#) with timestamp and user; does not persist across restarts under the chosen

Appendix: QC methodology detail

`check_constant_value_across_visits` — per-patient dominance ratio: share of patients for whom a value is invariant across *all* their visits, vs. expectation if personal-constant values were spread evenly across observed distinct values. Requires ≥ 2 visits, ≥ 10 subjects.

`check_value_spike` — population-level share of *every* record for a measure landing on one value, vs. the local background set by its nearest numeric neighbors.

Complementary to the check above: catches a filler used on only some of a patient's visits, which the per-patient check is blind to.

`check_guideline_range` — generic, reusable against any

Appendix: Future-work gaps, expanded

Full detail (data fields involved, why each matters clinically) lives in [.scratch/HANDOFF.md](#), “What’s NOT done — real gaps, not polish”. These read as “*what would make this SSc-domain-aware rather than generic-longitudinal-EHR-browser-aware*” to a pulmonary-research committee specifically — not code-quality gaps.